

第6章 利用数组处理 批量数据

Part B

复习：数组的基本概念

✧ 数组 —— 一组有序数据的集合。

- ◆ 每个元素都属于同一个数据类型
- ◆ 下标代表数据在数组中的序号，从0开始。
- ◆ 数组可以是一维的或多维的。
- ◆ 用一个数组名和下标来唯一地确定数组中的元素。

复习：一维数组和二维数组

✧ 定义

类型符 数组名 [常量表达式] ;

类型说明符 数组名[常量表达式1][常量表达式2];

✧ 引用

小心下标越界!

数组名[下标]

数组名[下标1][下标2]

✧ 初始化

类型说明符 数组名[常量表达式]= {常量1,...,常量n};

- 给出的初始值个数应等于或小于数组的元素个数，未给初始值的那些元素初值为0！
- 二维数组可以分行赋初值，也可以不分行赋初值（按行存放）。
- 提供初始数据的，定义数组时仅第一维的长度可省略。

复习：一维数组和二维数组

✧ 经典例程

- ◆ 起泡排序算法：例6.3
- ◆ 矩阵转置：例6.4
- ◆ 打擂台算法：例6.5

6.3 字符数组

C语言中，常量有字符串型的，但变量却没有，字符串需存放在字符型数组中。

字符数组

✧ 字符型数据：以字符的ASCII代码存储在存储单元中，一般占一个字节，属于整数类型

✧ **字符数组**：用来存放字符数据的数组，数组中的每个元素存放一个字符。

◆ 定义字符数组的方法与定义数值型数组的方法类似

【例】 `char c[100];`

◆ 用于保存文本等，由于程序中经常需要处理字符序列或者各种文本，因此C语言为处理字符数组提供了特殊支持。

• `<stdio.h>`和`<string.h>`提供了一些常用的字符串处理函数

字符数组的定义

✧ 字符数组的定义方法和普通数组的定义方式相同。

【例】 `char c[11];`

`c[0]='I'; c[1]='_'; c[2]='a'; c[3]='m';`
`c[4]='_'; c[5]='h'; c[6]='a'; c[7]='p';`
`c[8]='p'; c[9]='y'; c[10]='\0';`

`c[0]c[1]c[2]c[3]c[4]c[5]c[6]c[7]c[8]c[9] c[10]`

I	_	a	m	_	h	a	p	p	y	\0
---	---	---	---	---	---	---	---	---	---	----

字符数组的初始化

✧ 对字符数组也可以采用普通数组的初始化方式。

✧ 但初值个数只能小于或等于定义的数组长度。

- ◆ 当小于时，其余的元素自动定为空字符（‘\0’）。
- ◆ 当等于时候，可以省略数组长度，根据个数确定数组长度。（但数组的最后没有了‘\0’，会有很多不良后果）
- ◆ 当大于时，则出现语法错误。

例如：char a[5]={'H', 'e', 'l', 'l', 'o'};

char b[]={'H', 'e', 'l', 'l', 'o'}; //b的长度为5

char c[10]={'c', ' ', 'p', 'r', 'o', 'g', 'r', 'a', 'm'};

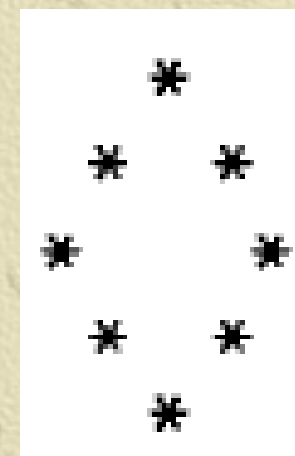
c[0]c[1]c[2]c[3]c[4]c[5]c[6]c[7]c[8]c[9]

c		p	r	o	g	r	a	m	\0
---	--	---	---	---	---	---	---	---	----

字符数组的初始化 (2)

✧ 也可以定义和初始化一个二维/多维字符数组

例如: `char diamond[5][5]={{' ',' ',' ',' ','*'},`
`{' ','*',' ',' ','*'},`
`{'*',' ',' ',' ','*'},`
`{' ','*',' ',' ','*'},`
`{' ',' ',' ',' ','*'} };`



例如: `char namelists[5][10]={{'A','p','p','l','e'},`
`{'D','e','l','l'}, {'I','B','M'}, {'S','o','n','y'},`
`{'S','u','n'}};`

字符数组的引用

✧ 可以引用字符数组中的一个元素，得到一个字符。

【例6.6】 输出一个已知的字符串。

【解题思路】

- ◆ 定义一个字符数组，并用“初始化列表”对其赋以初值
- ◆ 用循环逐个输出此字符数组中的字符

```
#include <stdio.h>
int main() 等价于char c[]=.....
{ char c[15] = {'I', ' ', 'a', 'm', ' ', 'a',
               ' ', 's', 't', 'u', 'd', 'e', 'n', 't', '.'};
  int i;
  for(i=0; i<15; i++)
    printf(" %c", c[i]);
  printf("\n");
  return 0;
}
```

试验：若用printf("%s", c);
会有何问题？

```
I am a student.
```

【例6.7】 输出一个菱形图。

【解题思路】

- ◆ 定义一个字符型的二维数组，用“初始化列表”进行初始化
- ◆ 用嵌套的for循环输出字符数组中的所有元素。

```

#include <stdio.h>
int main()
{ char diamond[][5] = { {' ', ' ', '*'},
                          {' ', '*', ' ', '*'}, {'*', ' ', ' ', '*'},
                          {' ', '*', ' ', '*'}, {' ', ' ', '*'} };

int i, j;
for (i = 0; i < 5; i++)
    { for (j = 0; j < 5; j++)
        printf(" %c", diamond[i][j]);
        printf("\n");
    }
return 0;

```

【思考】若不用数组怎么输出菱形图？

注意："和'是不同的，前者是空字符（编译报错），后者是空格。

```
#include <stdio.h>
```

```
int main()
```

```
{ int i,j,k;
```

```
-----
```

```
//上半空心菱形
```

```
for(i=1; i<=10; i++) { //控制行数 10
```

```
    for(k=10; k>i; k--) printf("□□"); //第i行前面有2*(10-i)个空格
```

```
    if(i != 1) printf("□*"); //从第二行起，每行左侧的*
```

```
    for(j=1; j<2*i-2; j++) printf("□□"); //第i行中心的2*(i-1)-1个空格
```

```
    printf("□*"); //每行右侧的*
```

```
    printf("\n");
```

```
}
```

[illegible]

}

字符串和字符串结束标志

✧ C语言中，将字符串作为字符数组来处理。

✧ 在实际应用中，往往关心的是字符串的**有效长度**而不是字符数组的长度。

例如：字符数组长度100，而实际有效字符只有40个。

✧ 为了测定字符串的实际长度，C语言规定了字符串结束标志‘\0’

例如：字符数组前9个字符都不是‘\0’，而第10个是，则有效字符个数为9。

- C程序往往依靠检测‘\0’的位置来判断字符串是否结束，而不是根据数组的长度。
- 定义字符数组时应保证数组长度始终大于字符串实际长度。

字符串和字符串结束标志（2）

✧ C系统在存储字符串常量时会自动加一个 ‘\0’ 作为结束符。

例如：“I am happy!”

I	_	a	m	_	h	a	p	p	y	!	\0
---	---	---	---	---	---	---	---	---	---	---	----

✧ 可以用字符串常量来初始化字符数组。

例如， `char c[ ]={"Hello"};`

或 `char c[]="Hello";`

[注意] 这样初始化后得到的字符数组的长度是6，即等价于 `char c[6]={'H','e','l','l','o','\0'};`，而不是5！

字符串和字符串结束标志（3）

例如，若有一个字符串中第一次出现 ‘\0’ 的是第12个字符（数组下标11），字符串的有效字符为11个。

“I am happy!\0ab”

I	_	a	m	_	h	a	p	p	y	!	\0	a	b
---	---	---	---	---	---	---	---	---	---	---	----	---	---

✱ 字符数组并不要求最后一个字符必须为 ‘\0’，但是为了处理方法一致，且便于检测字符串的实际长度，以及程序中作相应的处理，常常人为地加上一个 ‘\0’。

字符数组的输入输出

1. 逐个字符输入输出：对字符数组引用，并用 “%c” 输入或输出一个字符。

例如： `char c[ ] = "China";`
`printf("%c", c[3]);`

2. 整个字符串输入或输出：用 “%s” 格式符。

1) 输出

例如： `char c[ ] = "China";` 注意：若是 `char c[5] = "China";` 在实际输出时可能会有问题！
`printf("%s", c);`

- ◆ 输出字符中不包括结束符 ‘\0’;
- ◆ `printf` 的输出项是字符数组名称;
- ◆ 输出在遇到第一个 ‘\0’ 时停止。

例如： `char c[ ] = "China\0abcd";`
`printf("%s", c);`

字符数组的输入输出（2）

2) 输入

例如: `char c[10];`
`scanf("%s", c);` 前面不需要加&符

- ✧ 系统会自动在接受到的字符后面加一个 ‘\0’（用于接收的字符数组要足够大！）；
- ✧ 用一个scanf同时输入多个字符串时，各个输入串之间以空格分隔
- ✧ 用scanf输入的字符串中不能包含空格！

例如: `char str[20];`
`scanf("%s", str);` 输入” Good morning! ✓”

- ✧ scanf的输入项是字符数组名，不必加上&！（因为C语言中数组名代表数组的起始地址。）

```
char str1[5],str2[5],str3[5];  
scanf(" %s %s %s",str1,str2,str3);
```

How are you? ↙

str1	H	o	w	\0	\0
str2	a	r	e	\0	\0
str3	y	o	u	?	\0

使用字符串处理函数

✧ 为方便字符序列和文本的处理，几乎所有版本的C语言在函数库中都提供了处理字符串的函数。

常用的有：

- ◆ 字符串输入输出puts、gets
- ◆ 字符串连接strcat
- ◆ 字符串复制strcpy、strncpy
- ◆ 字符串比较strcmp
- ◆ 字符串长度strlen
- ◆ 字符串大小写strlwr、strupr

字符串处理函数

◆ puts

#include <stdio.h>

int puts (const char * s);

将一个字符串（以'\0'结束的字符序列）输出到终端。

- 输出时，将字符串结束标志'\0'转换成'\n'，即输出后自动换行。

（与printf的"%s"区别就在于此！）

例如：char a[]="china"; puts(a);

字符串处理函数

◆ gets

#include <stdio.h>

char * gets (char * string);

从终端输入一个字符串到字符数组，并且将该字符数组的起始地址作为返回值。

- 输入的结束标志是回车，最后换行符将转化为 ‘\0’。
- 与scanf的区别是其能接受带空格的字符串。

例如：char a[10]; gets(a);

字符串处理函数

◆ strlen (string length)

```
#include <string.h>
```

```
size_t strlen (const char * s);
```

测试字符串长度，不计 ‘\0’。

- 字符串长度 (K&R, P30)

```
int strlen( char s[])
```

```
{ int i;
```

```
  i=0;
```

```
  while (s[i]!='\0')
```

```
    ++i;
```

```
  return i;
```

```
}
```

字符串处理函数

◆ strcpy (string copy)

#include <string.h>

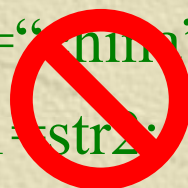
char * strcpy (char * dest, const char * src);

将第二个字符串复制到第一个字符数组中。

- 第一个字符数组必须定义得足够大。
- dest必须写成数组名的形式，src可以是数组名也可以是字符串常量。
- 不能用赋值语句将一个字符串常量或字符数组直接给一个字符数组。赋值语句只能适用对字符数组元素进行赋值。

错例:

```
str="nmn";  
str1=str2;
```



字符串复制 (K&R, P21)

void copy(char to[], char from[])

{ int i;

 i=0;

 while ((to[i]=from[i]) != '\0')

 ++i; 赋值，而不是判等！

}

头文件: #include<string.h>

char*strncpy(char*dest,char*src,size_tn);

✧ 复制字符串src中的内容到字符串dest中，复制多少由size_tn的值决定。

- ◆ 如果src的前n个字节不含NULL字符，则结果不会以NULL字符结束。如果src的长度小于n个字节，则以NULL填充dest直到复制完n个字节。
- ◆ src和dest所指内存区域不可以重叠且dest必须有足够的空间来容纳src的字符长度+'\0'。

✧ strcpy只是复制字符串，但不限制复制的字符数量，很容易造成缓冲溢出。strncpy要安全一些！

字符串处理函数

◆ **strcat** (**string** **catenate**)

#include <string.h>

char * strcat (char * dest, const char * src);

连接两个字符数组中的字符串，将字符串2接到字符串1之后，并将结果放在字符串1中。

- 要注意**第一个字符数组必须定义得足够大！**
- 原来两个字符串的结束标志，只保留第二个串的。

例如: **char str1[20]="Hello!";**

char str2[10]="Mike";

strcat(str1, str2);

字符串连接 (K&R, P38)

```
void strcat(char s[], char t[])
{ int i, j;
  i=j=0;
  while (s[i] != '\0') /*判断是否为字符串s尾部*/
    i++;
  while ((s[i++] = t[j++]) != '\0'); /*拷贝t */
}
```

赋值，而不是判等！

字符串处理函数

字典序

◆ **strcmp** (string compare)

#include <string.h>

int strcmp (const char * s1, const char * s2);

比较字符串，其比较规则就是，自左向右逐个比较字符（按ASCII码值大小比较），直到出现不同的字符或遇到'\0'。全部相同则返回0，否则返回第一对不同字符之间的差值。

例如：strcmp("china", "chin")返回值为97;

- 比较字符串时，不能将两个字符串名直接用比较运算符比较。

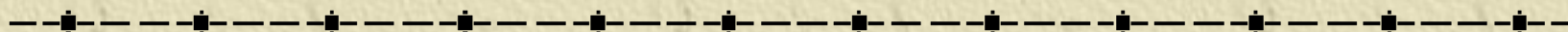
错例：if (str1==str2) printf("yes");

char str1[10], str2[] = "China";

str1 = "China"; 错误

str1 = str2; 错误

字典序:



"A"<"B" "a">"A"

"computer">"compare"

"these">"that" "1A">"\$20"

"CHINA">"CANADA"

"DOG"<"cat"

"Tsinghua">"TSINGHUA"

字符串比较 (K&R)

```
int strcmp(char *s, char *t)
{
    int i;
    for (i = 0; s[i] == t[i]; i++)
        if (s[i] == '\0')
            return 0;
    return s[i] - t[i];
}
```

字符串处理函数

- ◆ strlwr (string lowercase)

```
#include <string.h>
```

```
char * strlwr (char * s);
```

将字符串中的大写字母转换成小写字母。

- ◆ strupr (string uppercase)

```
#include <string.h>
```

```
char * strupr (char * s);
```

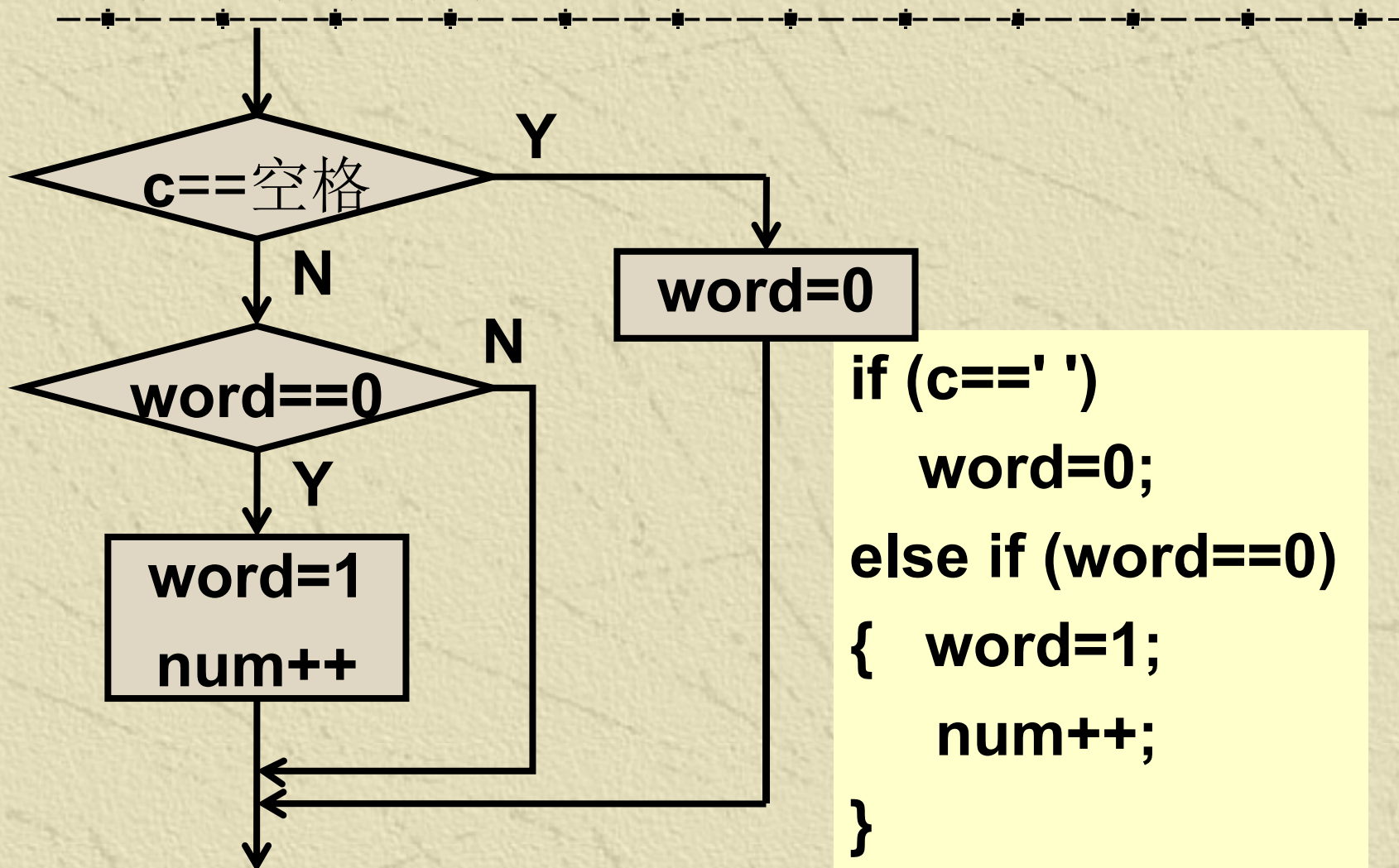
将字符串中的小写字母转换成大写字母。

【例6.8】 输入一行字符，统计其中有多少个单词，单词之间用空格分隔开。

【解题思路】 问题的关键是怎样确定“出现一个新单词了”

- ◆ 从第1个字符开始逐个字符进行检查，判断此字符是否是新单词的开头，如果是，就使变量num的值加1，最后得到的num的值就是单词总数
- ◆ 判断是否出现新单词，可以由是否有空格出现来决定(连续的若干个空格作为出现一次空格；一行开头的空格不统计在内)
- ◆ 如果测出某一个字符为非空格，而它的前面的字符是空格，则表示“新的单词开始了”，此时使num累加1；
- ◆ 如果当前字符为非空格而其前面的字符也是非空格，则num不应再累加1

- 用变量word作为判别当前是否开始了一个新单词的标志，若word=0表示未出现新单词，如出现了新单词，就把word置成1



- ◆ 用变量word作为判别当前是否开始了一个新单词的标志，若word=0表示未出现新单词，如出现了新单词，就把word置成1
- ◆ 前面一个字符是否空格可以从word的值看出来，若word等于0，则表示前一个字符是空格；如果word等于1，意味着前一个字符为非空格

当前字符	I		a	m		a		b	o	y	.
是否空格	否	是	否	否	是	否	是	否	否	否	否
word原值	0	1	0	1	1	0	1	0	1	1	1
新单词开始否	是	否	是	否	否	是	否	是	否	否	否
word新值	1	0	1	1	0	1	0	1	1	1	1
num值	1	1	2	2	2	3	3	4	4	4	4

.....

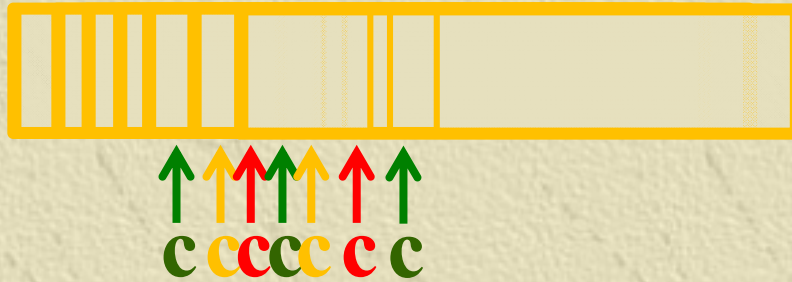
```
char string[81],c; int i,num=0,word=0;  
gets(string);  
for (i=0;(c=string[i])!='\0';i++)  
    if (c==' ') word=0;  
    else if (word==0)  
    { word=1;  
      num++;  
    }  
printf(" %d words\n",num);
```

.....

```
I am a boy.  
4 words
```

一定要设初始值

相当于
`c=string[i], c!='\0'`



.....

```
int i,num=0,word=0;
```

.....

```
for (i=0;(c=string[i])!='\0';i++)
```

```
    if (c==' ') word=0;
```

```
    else if (word==0)
```

```
    { word=1;
```

```
      num++;
```

```
    }
```

.....

It is OK!

.....

```
int i,num=0,word=0;
```

.....

```
for (i=0;string[i]!='\0';i++)  
    if (string[i]!=' ' &&  
        (i==0 || string[i-1]==' '))  
        num++;
```

.....

【例6.9】 有3个字符串,要求找出其中最大者。

【解题思路】 设一个二维的字符数组str,大小为 3×10 。每一行存放一个字符串

```
char str[3][10];
```

可以把str[0],str[1],str[2]看作3个一维字符数组,可以把它们如同一维数组那样进行处理

```
for (i=0;i<3;i++)
```

```
    gets (str[i]);
```

```
China
Japan
India
```

str[0]

C

h

i

n

a

\0

\0

\0

\0

\0

str[1]

J

a

p

a

n

\0

\0

\0

\0

\0

str[2]

I

n

d

i

a

\0

\0

\0

\0

\0

```
strcpy(string,str[0]);
```

打擂台

```
if (strcmp(str[2],string)>0)
```

```
strcpy(string,str[2]);
```

```
#include <stdio.h>
#include <string.h>
int main ( )
{char str[3][10]; char string[10]; int i;
  for (i=0;i<3;i++) gets (str[i]);
  if (strcmp(str[0],str[1])>0)
    strcpy(string,str[0]);
  else
    strcpy(string,str[1]);
  if (strcmp(str[2],string)>0)
    strcpy(string,str[2]);
  printf("\nthe largest:\n%s\n",string);
  return 0;
}
```

China

Japan

India

the largest:

Japan

```
#include <stdio.h>
#include <string.h>
int main( )
{ char str[3][10];
  int max=0, i;
  for (i=0; i<3; i++)
    gets (str[i]);
  for (i=1; i<3; i++)
    if (strcmp(str[i],str[max])>0)
      max = i;
  printf("\nthe largest:\n%s\n",
        str[max]);
  return 0;
}
```